

**INTERNATIONAL CYBERSECURITY  
AND DIGITAL FORENSICS ACADEMY**

**INTRO TO CRITICAL INFRASTRUCTURE  
FOR IT LEARNERS**

**LAB 03 — BUILDING YOUR FIRST PLC PROGRAM**

**Student Edition**

Authorized ICDFA training environments only  
[icdfa.edu.ng](http://icdfa.edu.ng)

## Lab 03: Building Your First PLC Program

**Course:** Intro to Critical Infrastructure for IT Learners | **Estimated Duration:** 4–5 hours

**Submission deadline:** As published in ICDFA E-Campus.

**Authorization & Safety:** Perform all activities only on the ICDFA-provided virtual machines, packet captures and isolated training networks. Do not bridge these VMs to production networks, scan external systems, or reuse credentials observed in training data.

### Learning Outcomes

- Create a new OpenPLC Ladder Diagram project.
- Define Boolean and Real variables for an industrial tank-fill process.
- Build logic to automatically fill a 0–10 ft tank between operating setpoints.
- Implement high/low alarm logic and an emergency-stop condition.
- Document and validate the intended control behavior before connecting to an HMI.

### Laboratory Environment

| System | Expected Address | Account | Password |
|--------|------------------|---------|----------|
| PLC VM | 192.168.1.10     | plc     | password |

**Note:** If the instructor-provided VM image uses updated addressing or credentials, use the values published in E-Campus. Do not alter addressing unless instructed.

### Variable Plan

| Variable          | Data Type | Purpose                                                  |
|-------------------|-----------|----------------------------------------------------------|
| TankFillAuto_Cmd  | BOOL      | Internal automatic fill latch/command                    |
| TankValveOpen     | BOOL      | Discrete command/status for the fill valve               |
| Estop_Cmd         | BOOL      | Emergency-stop input; TRUE forces automatic fill to stop |
| TankLevelHi_Alarm | BOOL      | High-level alarm state                                   |
| TankLevelLo_Alarm | BOOL      | Low-level alarm state                                    |
| TankLevel_LIT     | REAL      | Measured tank level in feet                              |
| TankEmpty_SP      | REAL      | Automatic fill start setpoint (2.0 ft)                   |
| TankFull_SP       | REAL      | Automatic fill stop setpoint (8.0 ft)                    |
| TankLoAlarm_SP    | REAL      | Low-level alarm setpoint (1.0 ft)                        |
| TankHiAlarm_SP    | REAL      | High-level alarm setpoint (9.0 ft)                       |

### Pre-Lab Checklist

- Read the complete lab before beginning.
- Confirm you can access the required VM(s) and that virtualization support is enabled.
- Use the instructor-specified isolated/host-only network.
- Create or restore the clean starting snapshot for this lab.
- Prepare a report document with your name, registration number, course, lab number and date.
- Capture evidence as you progress rather than repeating the lab at the end.

### Procedure

**Step 1.** Start the PLC VM from a clean snapshot. Launch OpenPLC Editor and create a new project folder named TankFill.

**Step 2.** Create a new POU named TankFill and select LD (Ladder Diagram) as the language.

**Step 3.** Add the variables listed in the Variable Plan table in this manual. Use BOOL for discrete states/commands and REAL for tank-level values and setpoints.

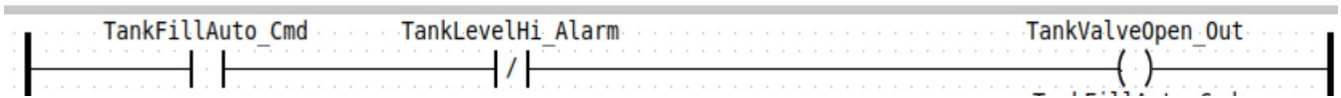
| program0     |                      |                   |      |          |               |        |                                                          |
|--------------|----------------------|-------------------|------|----------|---------------|--------|----------------------------------------------------------|
| Description: |                      | Class Filter: All |      |          |               |        |                                                          |
| #            | Name                 | Class             | Type | Location | Initial Value | Option | Documentation                                            |
| 1            | TankFillAuto_Cmd     | Local             | BOOL |          |               |        | Internal automatic fill latch/command                    |
| 2            | TankValveOpen        | Local             | BOOL |          |               |        | Discrete command/status for the fill valve               |
| 3            | EStop_Cmd            | Local             | BOOL |          |               |        | Emergency-stop input; TRUE forces automatic fill to stop |
| 4            | TankLevelHi_Alarm    | Local             | BOOL |          |               |        | High-level alarm state                                   |
| 5            | TankLevelLo_Alarm    | Local             | BOOL |          |               |        | Low-level alarm state                                    |
| 6            | TankLevel_LIT        | Input             | REAL |          |               |        | Measured tank level in feet                              |
| 7            | TankEmpty_SP         | Input             | REAL |          |               |        | Automatic fill start setpoint (2.0 ft)                   |
| 8            | TankFull_SP          | Input             | REAL |          |               |        | Automatic fill stop setpoint (8.0 ft)                    |
| 9            | TankLevelHi_Alarm_SP | Local             | BOOL |          |               |        | High-level alarm setpoint (9.0 ft)                       |
| 10           | TankLevelLoAlarm_SP  | Local             | BOOL |          |               |        | Low-level alarm setpoint (1.0 ft)                        |

**Step 4.** Set the operating values: TankEmpty\_SP = 2.0 ft, TankFull\_SP = 8.0 ft, TankLoAlarm\_SP = 1.0 ft and TankHiAlarm\_SP = 9.0 ft.

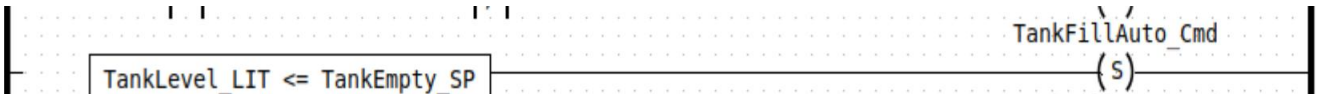
|    |                      |       |      |       |                                        |
|----|----------------------|-------|------|-------|----------------------------------------|
| 7  | TankEmpty_SP         | Input | REAL | 2.0ft | Automatic fill start setpoint (2.0 ft) |
| 8  | TankFull_SP          | Input | REAL | 8.0ft | Automatic fill stop setpoint (8.0 ft)  |
| 9  | TankLevelHi_Alarm_SP | Local | BOOL | 9.0ft | High-level alarm setpoint (9.0 ft)     |
| 10 | TankLevelLoAlarm_SP  | Local | BOOL | 1.0ft | Low-level alarm setpoint (1.0 ft)      |

**Step 5.** Insert left and right power rails with enough pins for the complete program, then create the first logic rung.

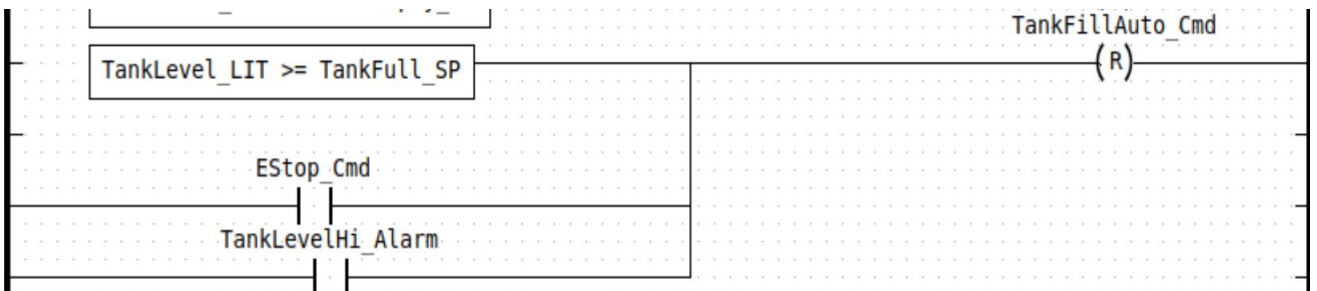
**Step 6.** Build the valve-control rung: TankFillAuto\_Cmd AND NOT TankLevelHi\_Alarm energizes TankValveOpen.



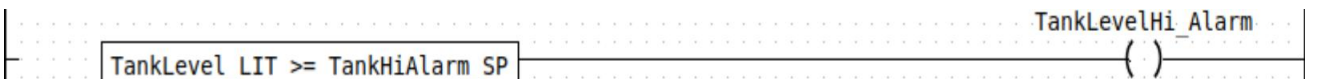
**Step 7.** Build the automatic-start rung: when TankLevel\_LIT <= TankEmpty\_SP, SET TankFillAuto\_Cmd.



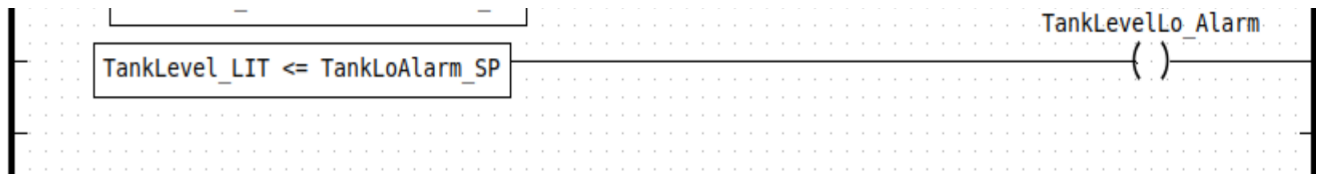
**Step 8.** Build the automatic-stop/safety rung: when TankLevel\_LIT >= TankFull\_SP OR EStop\_Cmd is TRUE OR TankLevelHi\_Alarm is TRUE, RESET TankFillAuto\_Cmd.



**Step 9.** Build the high-level alarm rung: TankLevel\_LIT >= TankHiAlarm\_SP energizes TankLevelHi\_Alarm.



**Step 10.** Build the low-level alarm rung: TankLevel\_LIT <= TankLoAlarm\_SP energizes TankLevelLo\_Alarm.



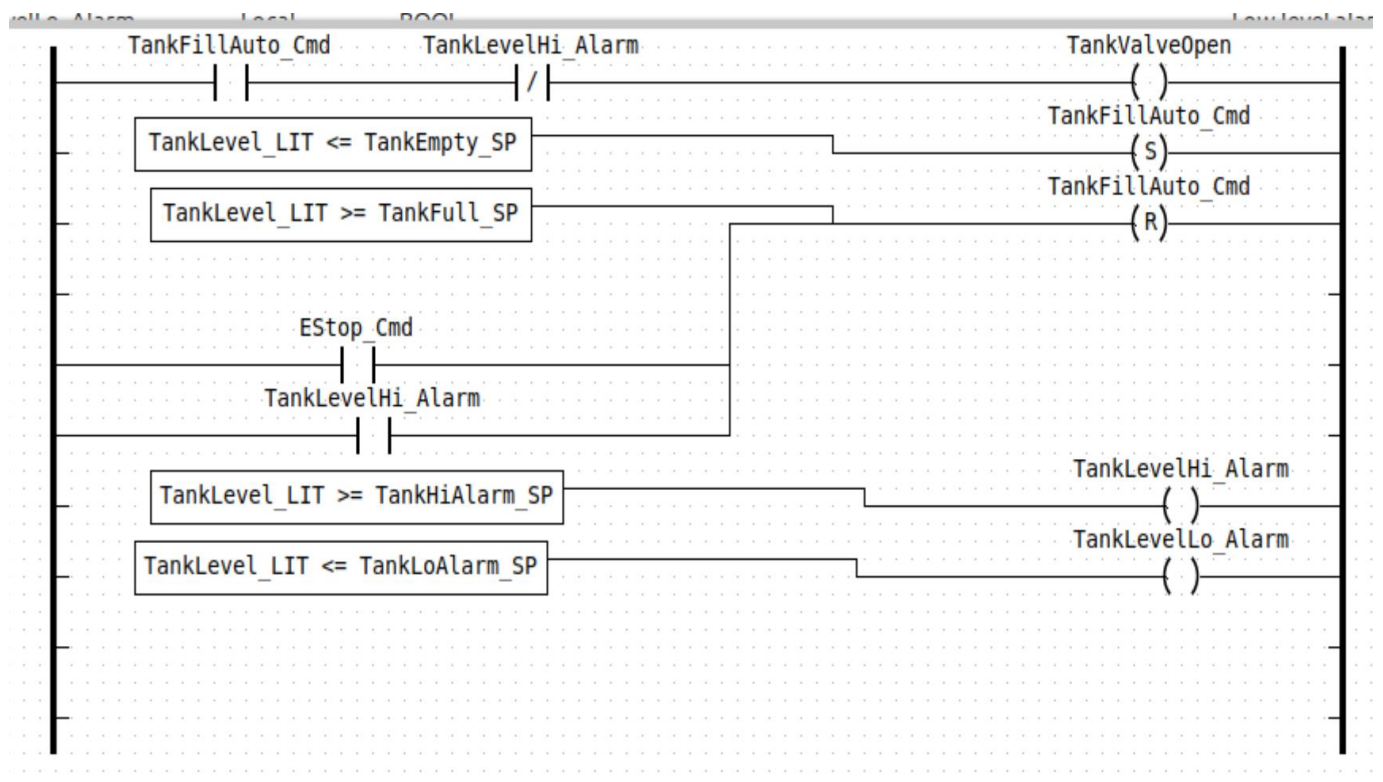
**Step 11.** Check every variable name and data type. Confirm that all contacts/coils/expressions are wired to complete rungs and that no instruction is floating.

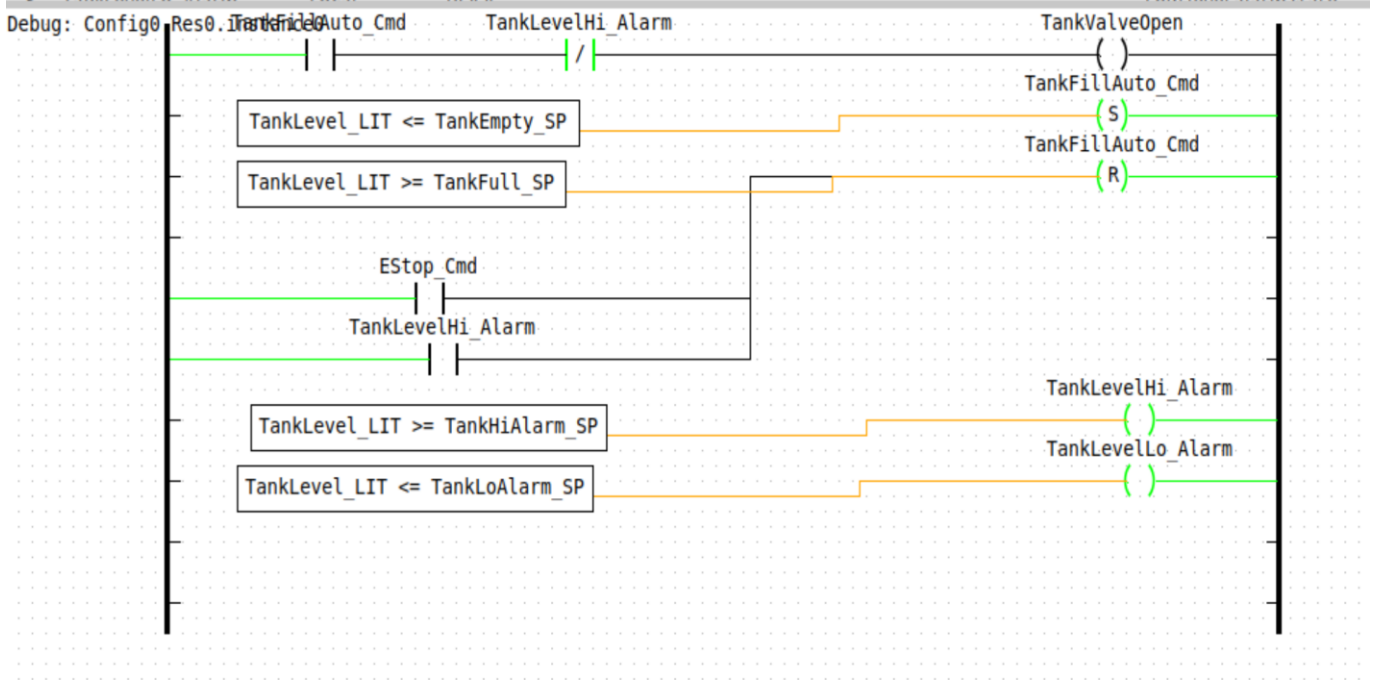
**Step 12.** Save the project. If the editor provides validation/compile checking in your VM image, run it and correct any errors before submission.

**Step 13.** Create a logic test table covering at least six scenarios: empty tank, normal filling, full tank, high alarm, low alarm and emergency stop.

| Scenario          | TankLevel_LIT | TankFillAuto_Cmd | TankLevelHi_Alarm | TankLevelLo_Alarm | TankValveOpen |
|-------------------|---------------|------------------|-------------------|-------------------|---------------|
| 1. Empty Tank     | 0.5 ft        | TRUE             | FALSE             | TRUE              | TRUE          |
| 2. Normal Filling | 3.0 ft        | TRUE             | FALSE             | FALSE             | TRUE          |
| 3. Full Tank      | 8.5 ft        | FALSE            | FALSE             | FALSE             | FALSE         |
| 4. High Alarm     | 9.5 ft        | FALSE            | TRUE              | FALSE             | FALSE         |
| 5. Low Alarm      | 0.8 ft        | TRUE             | FALSE             | TRUE              | TRUE          |
| 6. Emergency Stop | Any level     | FALSE            | FALSE             | FALSE             | FALSE         |

**Step 14.** Capture screenshots of the variable table and the completed ladder logic. Submit the project folder together with your written report.





### Required Evidence & Submission

- Screenshot of the completed variable/tag table.
- Screenshot(s) of the complete ladder program.
- A six-scenario logic test table showing inputs/conditions and expected outputs.
- The exported/saved TankFill OpenPLC project files.
- A short safety explanation describing why an emergency-stop condition must override normal automatic control.

**Recommended filename:** : C11/26/ACIS\_17347\_LAB03\_Building\_Your\_First\_PLC\_Program.pdf plus the required project/capture file(s).

### Knowledge Check

**1. Why are BOOL and REAL appropriate data types for this project?**

Response: \_BOOL is a binary data type that have only 2 values 0 or 1 while REAL is a data type allows for real numbers which can be negative or positive.\_

**2. What is the purpose of a SET/RESET pair for TankFillAuto\_Cmd?**

Response: \_It creates a latching (seal-in) circuit which allows the auto-fill command to stay active even after the initial trigger condition is gone.\_

**3. What should happen if the tank reaches 9 ft while the fill valve remains commanded open?**

Response: Rung 5 activates and the TanklevelHi\_Alaarm energizes which will break Rung 1 and close the valve. The high alarm acts as a safety override and forces the valve closed to prevent overflow, regardless of the auto-fill command.

**4. Why should PLC logic be validated in simulation before use with physical equipment?**

Response: \_Simulation allows engineers to catch logic errors, test edge cases, and verify behavior before anything expensive or dangerous happens in the real world.\_

### Marking Rubric — 100 Marks

| Criterion           | Marks | Standard                                                                              |
|---------------------|-------|---------------------------------------------------------------------------------------|
| Technical execution | 50    | Lab procedure completed correctly; required configuration/analysis works as intended. |
| Evidence            | 20    | Screenshots/files are complete, readable, relevant and clearly labelled.              |

**INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY (ICDFA)**  
INTRO TO CRITICAL INFRASTRUCTURE FOR IT LEARNERS

| Criterion                                                                                                                                                                                                                            | Marks | Standard                                                                                      |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|-----------------------------------------------------------------------------------------------|
| Analysis & knowledge check                                                                                                                                                                                                           | 20    | Responses demonstrate understanding of the industrial-control concepts observed.              |
| Professional reporting                                                                                                                                                                                                               | 10    | Clear structure, correct terminology, concise writing, file naming and submission compliance. |
| <b>Submission Integrity:</b> Evidence must come from the student's own assigned laboratory environment. Screenshots should show enough context to verify the activity. Do not submit another learner's screenshots or project files. |       |                                                                                               |